

Blueprint Subsea • Oculus M750D

Running the Sonar & Building 2D Sonar SLAM on Ubuntu

Complete step-by-step guide: SDK setup → Point cloud extraction → 2D SLAM with sonar-SLAM (ROS/GTSAM)



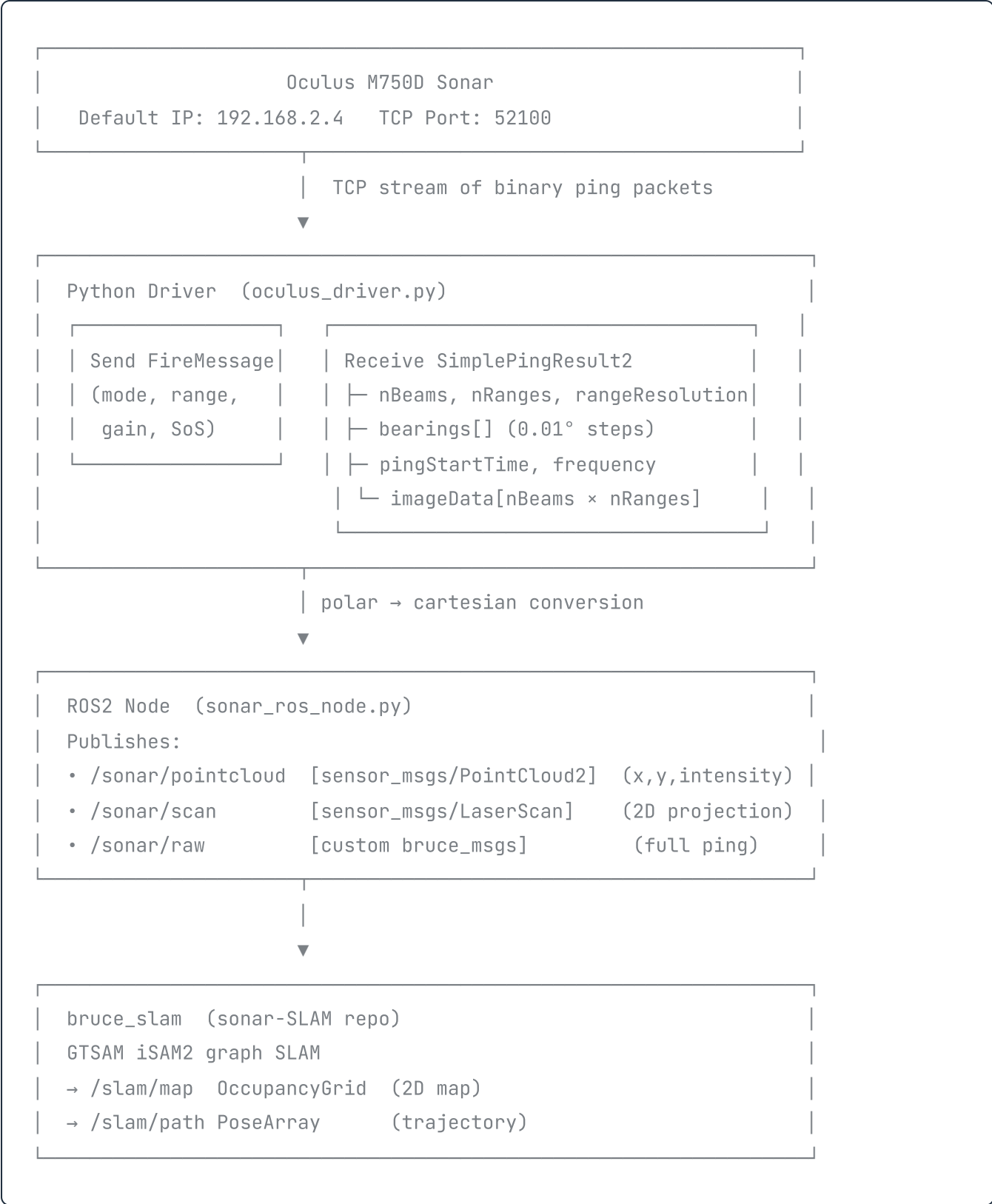
CONTENTS

1. Understanding the Data Flow
2. System Prerequisites
3. Python Sonar Driver (raw TCP)
4. ROS2 Sonar Node — Point Cloud + Intensity Publisher
5. 2D SLAM Setup using sonar-SLAM repo
6. Launch & Visualize Everything
7. Troubleshooting Reference

SECTION 01

Understanding the Data Flow

The Oculus M750D is a dual-frequency multibeam imaging sonar. It communicates over standard TCP/IP — no special kernel drivers needed. Here's how data moves end-to-end:



Key Data Structures (from Oculus.h SDK)

Everything you extract from the sonar comes packed in these two C structs:

FIELD	TYPE	MEANING
nBeams	uint16	Number of angular beams per ping (up to 512 on M750d)

nRanges	uint16	Number of range bins (samples along each beam)
rangeResolution	double (m)	Distance represented by one range bin
bearings[]	int16[] × 0.01°	Angle of each beam in hundredths of a degree
pingStartTime	double (s)	Seconds since sonar power-on, microsecond precision
frequency	double (Hz)	Acoustic frequency used (750 kHz or 1.2 MHz for M750d)
imageData	uint8[] or uint16[]	Intensity values, row = range bin, col = beam

The image is stored in **polar coordinates**: `image[range_bin][beam_index]`. You convert to Cartesian x,y using:

```
angle_rad = bearing_deg * π / 180
range_m   = range_bin_index * rangeResolution

x = range_m * sin(angle_rad) ← horizontal (port/starboard)
y = range_m * cos(angle_rad) ← forward
```

MATH

SECTION 02

System Prerequisites

 **Recommended OS:** Ubuntu 22.04 LTS. The sonar-SLAM repo targets ROS2 Humble (ships with Ubuntu 22.04). All commands below assume Ubuntu 22.04 + ROS2 Humble.

Step 1 — Network Setup

The M750D ships with a fixed IP. Connect it directly (or via a switch) to your Ubuntu machine's Ethernet port.

1.1

Assign a static IP to your Ubuntu NIC on the same subnet as the sonar:

```
# Replace eth0/enp3s0 with your actual interface name (ip link show)
sudo ip addr add 192.168.2.100/24 dev enp3s0
```

BASH

```
sudo ip link set enp3s0 up

# Verify you can ping the sonar (default sonar IP: 192.168.2.4)
ping 192.168.2.4
```

1.2

Confirm the sonar's TCP port is reachable:

```
nc -zv 192.168.2.4 52100
# Expected: Connection to 192.168.2.4 52100 port [tcp] succeeded!
```

BASH



If the sonar has a different IP: Use the official Oculus Viewer (Windows/Mac) to find and reconfigure it first, or use ARP scan: `sudo arp-scan --localnet`

Step 2 — Install System Dependencies

```
# Python deps for the driver
sudo apt update
sudo apt install -y python3-pip python3-numpy python3-scipy

pip3 install numpy scipy matplotlib

# ROS2 Humble (skip if already installed)
sudo apt install -y ros-humble-desktop ros-humble-sensor-msgs \
  ros-humble-geometry-msgs ros-humble-nav-msgs \
  ros-humble-tf2-ros ros-humble-tf2-geometry-msgs \
  python3-colcon-common-extensions

# GTSAM (required by bruce_slam)
pip3 install gtsam

# Additional SLAM deps
sudo apt install -y ros-humble-slam-toolbox \
```

BASH

```
ros-humble-robot-localization \  
ros-humble-rviz2
```

Step 3 — Clone sonar-SLAM repo

BASH

```
mkdir -p ~/ros2_ws/src  
cd ~/ros2_ws/src  
git clone https://github.com/jake3991/sonar-SLAM.git  
  
# The repo contains 3 packages:  
# bruce/          → top-level launch files  
# bruce_msgs/     → custom ROS2 message types  
# bruce_slam/     → core SLAM node (GTSAM iSAM2)
```

SECTION 03

Python Sonar Driver

This is a standalone Python module that handles the raw TCP connection to the sonar, sends fire commands, receives ping data, and parses the binary packet format defined in `Oculus.h`.

Create this file at `~/ros2_ws/src/oculus_driver/oculus_driver.py`:

PYTHON

```
#!/usr/bin/env python3  
"""  
Oculus M750D Python Driver  
Implements the Oculus SDK communication protocol from Oculus.h (SDK v1.15.168)  
  
Protocol:  
- TCP port: 52100  
- Send: OculusSimpleFireMessage2 (fire command)  
- Receive: OculusSimplePingResult2 (ping data + image)  
- Image layout: [nRanges rows × nBeams cols], uint8 or uint16 intensity  
"""  
  
import socket
```

```

import struct
import numpy as np
import threading
import time
from dataclasses import dataclass
from typing import Optional, Callable

# -----
# Protocol constants (from Oculus.h)
# -----

OCULUS_CHECK_ID      = 0x4f53
MSG_SIMPLE_FIRE      = 0x15
MSG_SIMPLE_PING_RESULT = 0x23
MSG_DUMMY            = 0xff
DEFAULT_PORT         = 52100

DATA_SIZE_8BIT  = 0
DATA_SIZE_16BIT = 1

PING_RATE_NORMAL = 0x00    # ~10 Hz
PING_RATE_HIGH   = 0x01    # ~15 Hz
PING_RATE_HIGHEST = 0x02   # ~40 Hz

# -----
# OculusMessageHeader (14 bytes, packed)
# -----

HEADER_FMT = '<HHHHHHIH'    # little-endian
HEADER_SIZE = struct.calcsize(HEADER_FMT) # 16 bytes

# OculusSimpleFireMessage2 body (after header)
# masterMode(B) pingRate(B) netSpeed(B) gamma(B) flags(B) range(d)
# gainPercent(d) sos(d) salinity(d) extFlags(I) reserved0[2](II)
# beaconFreq(I) reserved1[5](IIIII)
FIRE_BODY_FMT = '<BBBBBdddIIIIIIIII'
FIRE_BODY_SIZE = struct.calcsize(FIRE_BODY_FMT)

# OculusSimplePingResult2 body (after header + fire message)

```

```

# pingId(I) status(I) freq(d) temp(d) pressure(d)
# heading(d) pitch(d) roll(d) sos_used(d) pingStartTime(d)
# dataSize(B) rangeRes(d) nRanges(H) nBeams(H)
# spare0-3(IIII) imageOffset(I) imageSize(I) messageSize(I)
PING_RESULT_FMT = '<IIdddddddbdHHIIIIII'
PING_RESULT_SIZE = struct.calcsize(PING_RESULT_FMT)

@dataclass
class SonarPing:
    """One complete ping frame from the sonar"""
    ping_id: int
    timestamp: float # seconds since sonar powerup
    frequency: float # Hz (750k or 1.2M)
    temperature: float # °C
    pressure: float # bar
    heading: float # degrees
    pitch: float
    roll: float
    speed_of_sound: float # m/s (actual used)
    range_resolution: float # m per range bin
    n_ranges: int
    n_beams: int
    bearings_deg: np.ndarray # shape (n_beams,), in degrees
    image: np.ndarray # shape (n_ranges, n_beams), float32 [0..1]

    def to_pointcloud(self, intensity_threshold: float = 0.05):
        """
        Convert polar image to Cartesian point cloud.

        Returns:
            points: np.ndarray shape (N, 2) columns = [x, y] in metres
            intensity: np.ndarray shape (N,) normalized [0..1]
            times: np.ndarray shape (N,) approximate timestamp per point
        """
        brg_rad = np.deg2rad(self.bearings_deg) # (n_beams,)
        ranges = np.arange(self.n_ranges) * self.range_resolution # (n_ranges,

```

```

# Meshgrid: rows=range, cols=bearing
R, B = np.meshgrid(ranges, brg_rad, indexing='ij')    # (n_ranges, n_bea

x = R * np.sin(B)    # port (-) / starboard (+)
y = R * np.cos(B)    # forward

mask = self.image > intensity_threshold

pts    = np.column_stack([x[mask], y[mask]])
intens = self.image[mask]

# Approx per-point time (linear across ping duration ~10ms)
t = np.full(pts.shape[0], self.timestamp)

return pts, intens, t

class OculusDriver:
    """
    Manages TCP connection to the Oculus sonar.
    Usage:
        driver = OculusDriver('192.168.2.4')
        driver.start(callback=my_fn)    # my_fn(ping: SonarPing)
        ...
        driver.stop()
    """

    def __init__(self, host: str, port: int = DEFAULT_PORT,
                  mode: int = 1, range_m: float = 10.0,
                  gain: float = 40.0, speed_of_sound: float = 1500.0,
                  salinity: float = 0.0):
        self.host      = host
        self.port      = port
        self.mode      = mode          # 1=750kHz  2=1.2MHz
        self.range_m   = range_m
        self.gain      = gain
        self.sos       = speed_of_sound
        self.salinity   = salinity

```

```
self._sock: Optional[socket.socket] = None
self._thread: Optional[threading.Thread] = None
self._running = False
self._callback: Optional[Callable] = None
self._rx_buf = bytearray()

def start(self, callback: Callable[[SonarPing], None]):
    self._callback = callback
    self._running = True
    self._thread = threading.Thread(target=self._run, daemon=True)
    self._thread.start()

def stop(self):
    self._running = False
    if self._sock:
        self._sock.close()

def _build_fire_message(self) → bytes:
    # Header
    body_size = FIRE_BODY_SIZE
    header = struct.pack(HEADER_FMT,
        OCULUS_CHECK_ID, # oculusId
        0,                # srcDeviceId
        0,                # dstDeviceId
        MSG_SIMPLE_FIRE, # msgId
        2,                # msgVersion
        body_size,        # payloadSize
        0                 # partNumber
    )
    # Body: OculusSimpleFireMessage2
    body = struct.pack(FIRE_BODY_FMT,
        self.mode,        # masterMode
        PING_RATE_NORMAL, # pingRate
        0,                # networkSpeed (0=unrestricted)
        127,              # gammaCorrection
        0x19,             # flags (gain assist off)
        self.range_m,
```

```

        self.gain,
        self.sos,
        self.salinity,
        0, 0, 0, 0,          # extFlags, reserved0[2], beaconFreq
        0, 0, 0, 0, 0       # reserved1[5]
    )

    return header + body

def _run(self):
    while self._running:
        try:
            self._sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            self._sock.settimeout(5.0)
            self._sock.connect((self.host, self.port))
            print(f"[OculusDriver] Connected to {self.host}:{self.port}")

            # Send initial fire command
            self._sock.sendall(self._build_fire_message())
            self._rx_buf.clear()

            while self._running:
                chunk = self._sock.recv(65536)
                if not chunk:
                    break

                self._rx_buf.extend(chunk)
                self._process_buffer()

            except Exception as e:
                print(f"[OculusDriver] Error: {e}, reconnecting in 2s...")
                time.sleep(2)
            finally:
                if self._sock:
                    self._sock.close()

def _process_buffer(self):
    while len(self._rx_buf) ≥ HEADER_SIZE:
        # Find OCULUS_CHECK_ID sync word 0x4f53
        sync_pos = -1

```

```
for i in range(len(self._rx_buf) - 1):
    if self._rx_buf[i] == 0x53 and self._rx_buf[i+1] == 0x4f:
        sync_pos = i
        break

if sync_pos < 0:
    self._rx_buf.clear()
    return

if sync_pos > 0:
    del self._rx_buf[:sync_pos]

if len(self._rx_buf) < HEADER_SIZE:
    return

hdr = struct.unpack_from(HEADER_FMT, self._rx_buf, 0)
oculus_id, src, dst, msg_id, msg_ver, payload_size, part_num = hdr

total_size = HEADER_SIZE + payload_size
if len(self._rx_buf) < total_size:
    return # wait for more data

pkt = bytes(self._rx_buf[:total_size])
del self._rx_buf[:total_size]

if msg_id == MSG_SIMPLE_PING_RESULT:
    ping = self._parse_ping(pkt)
    if ping and self._callback:
        self._callback(ping)

def _parse_ping(self, pkt: bytes) → Optional[SonarPing]:
    try:
        offset = HEADER_SIZE + FIRE_BODY_SIZE
        fields = struct.unpack_from(PING_RESULT_FMT, pkt, offset)

        (ping_id, status, freq, temp, pressure,
         heading, pitch, roll, sos_used, ping_start_time,
         data_size, range_res, n_ranges, n_beams,
         sp0, sp1, sp2, sp3,
```

```

        image_offset, image_size, msg_size) = fields

    # Bearings array: n_beams × int16, right after the result header
    brg_offset = offset + PING_RESULT_SIZE
    brg_raw = struct.unpack_from(f'<{n_beams}h', pkt, brg_offset)
    bearings_deg = np.array(brg_raw, dtype=np.float32) * 0.01

    # Image data
    img_start = image_offset    # relative to start of message
    img_end    = img_start + image_size

    if data_size == DATA_SIZE_8BIT:
        raw = np.frombuffer(pkt[img_start:img_end], dtype=np.uint8)
        image = raw.reshape((n_ranges, n_beams)).astype(np.float32) / 255
    else:
        raw = np.frombuffer(pkt[img_start:img_end], dtype=np.uint16)
        image = raw.reshape((n_ranges, n_beams)).astype(np.float32) / 65535

    return SonarPing(
        ping_id=ping_id,
        timestamp=ping_start_time,
        frequency=freq,
        temperature=temp,
        pressure=pressure,
        heading=heading,
        pitch=pitch,
        roll=roll,
        speed_of_sound=sos_used,
        range_resolution=range_res,
        n_ranges=n_ranges,
        n_beams=n_beams,
        bearings_deg=bearings_deg,
        image=image,
    )
except Exception as e:
    print(f"[OculusDriver] Parse error: {e}")
    return None

```

```

# -----
# Quick standalone test (no ROS needed)
# Run: python3 oculus_driver.py
# -----

if __name__ == '__main__':
    import matplotlib.pyplot as plt

    received = []

    def on_ping(ping: SonarPing):
        received.append(ping)
        pts, intens, _ = ping.to_pointcloud(intensity_threshold=0.1)
        print(f"Ping {ping.ping_id}: {ping.n_beams} beams × {ping.n_ranges} ranges | {len(pts)} points | T={ping.temperature:.1f}°C")
        if len(received) == 5:
            plt.figure(figsize=(10, 10))
            plt.scatter(pts[:, 0], pts[:, 1],
                        c=intens, cmap='hot', s=1, alpha=0.7)

            plt.axis('equal')
            plt.xlabel('X (m)'); plt.ylabel('Y (m)')
            plt.title('Oculus M750D Point Cloud')
            plt.colorbar(label='Intensity')
            plt.savefig('/tmp/sonar_pointcloud.png', dpi=150)
            print("Saved /tmp/sonar_pointcloud.png")

    driver = OculusDriver(
        host='192.168.2.4',
        mode=1,          # 1 = low frequency (750kHz), better range
        range_m=10.0,    # 10 metre range
        gain=40.0,       # 40% gain
        speed_of_sound=1500.0,
        salinity=0.0     # 0 = fresh water, 35 = seawater
    )
    driver.start(on_ping)

    try:
        while True:

```

```

        time.sleep(0.1)
    except KeyboardInterrupt:
        driver.stop()
        print("Stopped.")

```

Test the driver (no ROS needed)

BASH

```

python3 oculus_driver.py
# You should see:
# [OculusDriver] Connected to 192.168.2.4:52100
# Ping 1: 256 beams × 500 ranges | 3421 points | T=22.3°C
# Ping 2: ...
# After 5 pings: saves /tmp/sonar_pointcloud.png

```

SECTION 04

ROS2 Sonar Node

This ROS2 node wraps the driver and publishes three topics that the sonar-SLAM pipeline consumes.

Package Structure

```

~/ros2_ws/src/oculus_driver/ | ├── oculus_driver/ | ├── __init__.py | ├──
oculus_driver.py ← the driver from Section 03 | ├── sonar_ros_node.py ← this
section | ├── package.xml | ├── setup.py | ├── setup.cfg

```

Create package.xml

XML

```

<?xml version="1.0"?>
<package format="3">
  <name>oculus_driver</name>
  <version>1.0.0</version>
  <description>ROS2 driver for Oculus M750D sonar</description>
  <maintainer email="you@example.com">You</maintainer>
  <license>MIT</license>

```

```

<depend>roscpp</depend>
<depend>sensor_msgs</depend>
<depend>geometry_msgs</depend>
<depend>std_msgs</depend>
<export>
  <build_type>ament_python</build_type>
</export>
</package>

```

Create setup.py

PYTHON

```

from setuptools import setup

setup(
    name='oculus_driver',
    version='1.0.0',
    packages=['oculus_driver'],
    install_requires=['setuptools'],
    entry_points={
        'console_scripts': [
            'sonar_node = oculus_driver.sonar_ros_node:main',
        ],
    },
)

```

sonar_ros_node.py

PYTHON

```

#!/usr/bin/env python3
"""
ROS2 node: Oculus M750D → PointCloud2 + LaserScan + raw intensity
Topics published:
  /sonar/pointcloud  sensor_msgs/PointCloud2  (x, y, intensity)
  /sonar/scan        sensor_msgs/LaserScan    (2D projection for SLAM)
  /sonar/image        sensor_msgs/Image       (raw polar image, debug)
"""

import rclpy

```

```
from rclpy.node import Node
import numpy as np
import struct

from sensor_msgs.msg import PointCloud2, PointField, LaserScan, Image
from std_msgs.msg import Header
from builtin_interfaces.msg import Time

from oculus_driver.oculus_driver import OculusDriver, SonarPing

def secs_to_ros_time(secs: float) → Time:
    t = Time()
    t.sec = int(secs)
    t.nanosec = int((secs - t.sec) * 1e9)
    return t

def make_pointcloud2(pts: np.ndarray, intensity: np.ndarray,
                    header: Header) → PointCloud2:
    """Build a sensor_msgs/PointCloud2 from (N,2) xy points + (N,) intensity"""
    n = pts.shape[0]
    fields = [
        PointField(name='x', offset=0, datatype=PointField.FLOAT32, count=1),
        PointField(name='y', offset=4, datatype=PointField.FLOAT32, count=1),
        PointField(name='z', offset=8, datatype=PointField.FLOAT32, count=1),
        PointField(name='intensity', offset=12, datatype=PointField.FLOAT32, count=1)
    ]
    point_step = 16 # 4 floats × 4 bytes
    data = np.zeros((n, 4), dtype=np.float32)
    data[:, 0] = pts[:, 0] # x
    data[:, 1] = pts[:, 1] # y
    data[:, 2] = 0.0 # z = 0 (2D)
    data[:, 3] = intensity

    msg = PointCloud2()
    msg.header = header
    msg.height = 1
```

```

    msg.width          = n
    msg.fields          = fields
    msg.is_bigendian    = False
    msg.point_step      = point_step
    msg.row_step        = point_step * n
    msg.is_dense        = True
    msg.data            = data.tobytes()
    return msg

def make_laserscan(ping: SonarPing, header: Header,
                  intensity_threshold=0.05) → LaserScan:
    """
    Project sonar ping to a 2D LaserScan by taking the MAX intensity range
    bin along each beam (detects the first strong reflector per beam).
    This is what feeds 2D SLAM.
    """
    brg_rad = np.deg2rad(ping.bearings_deg)
    n_beams = ping.n_beams

    ranges_m = np.zeros(n_beams, dtype=np.float32)
    intensities = np.zeros(n_beams, dtype=np.float32)

    for b in range(n_beams):
        beam = ping.image[:, b]
        if beam.max() > intensity_threshold:
            # First range bin exceeding threshold (closest strong return)
            idx = np.argmax(beam > intensity_threshold)
            ranges_m[b] = idx * ping.range_resolution
            intensities[b] = beam[idx]

    msg = LaserScan()
    msg.header = header
    msg.angle_min = float(brg_rad.min())
    msg.angle_max = float(brg_rad.max())
    msg.angle_increment = float((brg_rad.max() - brg_rad.min()) / (n_beams - 1))
    msg.time_increment = 0.0
    msg.scan_time = 0.1

```

```
msg.range_min      = ping.range_resolution
msg.range_max      = ping.n_ranges * ping.range_resolution
msg.ranges          = ranges_m.tolist()
msg.intensities     = intensities.tolist()
return msg
```

```
class SonarNode(Node):
    def __init__(self):
        super().__init__('oculus_sonar_node')

        # Parameters (can be overridden on CLI)
        self.declare_parameter('sonar_ip',      '192.168.2.4')
        self.declare_parameter('sonar_mode',    1)
        self.declare_parameter('range_m',       10.0)
        self.declare_parameter('gain',          40.0)
        self.declare_parameter('speed_of_sound', 1500.0)
        self.declare_parameter('salinity',       0.0)
        self.declare_parameter('intensity_threshold', 0.05)
        self.declare_parameter('frame_id',      'sonar_link')

        ip = self.get_parameter('sonar_ip').value
        thr = self.get_parameter('intensity_threshold').value

        self._frame = self.get_parameter('frame_id').value
        self._thr = thr

        # Publishers
        self._pub_pc = self.create_publisher(PointCloud2, '/sonar/pointcloud',
        self._pub_ls = self.create_publisher(LaserScan, '/sonar/scan',
        self._pub_img = self.create_publisher(Image, '/sonar/image',

        # Start driver
        self._driver = OculusDriver(
            host=ip,
            mode=self.get_parameter('sonar_mode').value,
            range_m=self.get_parameter('range_m').value,
            gain=self.get_parameter('gain').value,
```

```
        speed_of_sound=self.get_parameter('speed_of_sound').value,
        salinity=self.get_parameter('salinity').value,
    )

    self._driver.start(self._on_ping)
    self.get_logger().info(f"Sonar node started, connecting to {ip}")

def _on_ping(self, ping: SonarPing):
    now = self.get_clock().now().to_msg()
    hdr = Header()
    hdr.stamp      = now
    hdr.frame_id = self._frame

    # 1) Point cloud
    pts, intens, _ = ping.to_pointcloud(self._thr)
    if pts.shape[0] > 0:
        pc_msg = make_pointcloud2(pts, intens, hdr)
        self._pub_pc.publish(pc_msg)

    # 2) LaserScan (for 2D SLAM)
    ls_msg = make_laserscan(ping, hdr, self._thr)
    self._pub_ls.publish(ls_msg)

    # 3) Raw polar image (for debugging in RViz)
    img_msg = Image()
    img_msg.header      = hdr
    img_msg.height      = ping.n_ranges
    img_msg.width       = ping.n_beams
    img_msg.encoding    = 'mono8'
    img_msg.step        = ping.n_beams
    img_msg.data        = (ping.image * 255).astype(np.uint8).tobytes()
    self._pub_img.publish(img_msg)

    self.get_logger().debug(
        f"Ping {ping.ping_id}: {pts.shape[0]} pts, "
        f"T={ping.temperature:.1f}°C, SoS={ping.speed_of_sound:.0f}m/s")

def destroy_node(self):
    self._driver.stop()
```

```
        super().destroy_node()

def main(args=None):
    rclpy.init(args=args)
    node = SonarNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Build the package

```
cd ~/ros2_ws
colcon build --packages-select oculus_driver
source install/setup.bash
```

BASH

SECTION 05

2D SLAM Setup — sonar-SLAM repo



About the repo: jake3991/sonar-SLAM implements graph-based SLAM with GTSAM iSAM2. It was designed for a BlueROV with a DVL + IMU + imaging sonar. For 2D-only SLAM (no DVL/IMU), we configure it with a scan-matching odometry fallback.

Step 1 — Build sonar-SLAM

BASH

```
cd ~/ros2_ws

# Install Python deps for bruce_slam
pip3 install gtsam numpy scipy open3d

# Build all packages
colcon build --symlink-install
source install/setup.bash
```

Step 2 — Add a TF (transform) broadcaster

SLAM needs a coordinate frame tree. Create

```
~/ros2_ws/src/oculus_driver/oculus_driver/tf_broadcaster.py:
```

PYTHON

```
#!/usr/bin/env python3
"""
Minimal static TF publisher:
    map → odom → base_link → sonar_link
For testing without a real robot, odom stays at origin (dead reckoning = 0).
"""
import rclpy
from rclpy.node import Node
from tf2_ros import StaticTransformBroadcaster
from geometry_msgs.msg import TransformStamped

class StaticTF(Node):
    def __init__(self):
        super().__init__('static_tf')
        br = StaticTransformBroadcaster(self)

    def make_tf(parent, child, x=0, y=0, z=0):
        t = TransformStamped()
        t.header.stamp = self.get_clock().now().to_msg()
        t.header.frame_id = parent
        t.child_frame_id = child
        t.transform.translation.x = x
        t.transform.translation.y = y
        t.transform.translation.z = z
```

```

        t.transform.rotation.w = 1.0
    return t

    br.sendTransform([
        make_tf('map',      'odom'),
        make_tf('odom',     'base_link'),
        make_tf('base_link', 'sonar_link', z=0.1),
    ])

def main(args=None):
    rclpy.init(args=args)
    node = StaticTF()
    rclpy.spin(node)

if __name__ == '__main__':
    main()

```

Step 3 — SLAM Toolbox config (2D)

Instead of bruce_slam's full 3D pipeline, we wire our `/sonar/scan` topic into **slam_toolbox** — a battle-tested 2D SLAM system in ROS2. Create

`~/ros2_ws/src/oculus_driver/config/slam_toolbox_params.yaml`:

```

slam_toolbox:
  ros__parameters:
    # — Core —
    solver_plugin: solver_plugins::CeresSolver
    ceres_linear_solver: SPARSE_NORMAL_CHOLESKY
    ceres_preconditioner: SCHUR_JACOBI
    ceres_trust_strategy: LEVENBERG_MARQUARDT

    # — Map —
    map_frame: map
    odom_frame: odom
    base_frame: base_link
    scan_topic: /sonar/scan      # ← your sonar LaserScan topic
    mode: mapping

```

YAML

```

# — Update policy —
map_update_interval: 2.0
resolution: 0.05           # 5 cm per cell
max_laser_range: 10.0      # match your sonar range

# — Scan matching —
minimum_travel_distance: 0.1
minimum_travel_heading: 0.1
use_scan_matching: true
use_scan_barycenter: true
minimum_time_interval: 0.1

# — Loop closure —
do_loop_closing: true
loop_search_distance: 3.0
loop_match_minimum_response_fine: 0.3

```

Step 4 — Launch file

Create `~/ros2_ws/src/oculus_driver/launch/slam.launch.py`:

PYTHON

```

from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
from ament_index_python.packages import get_package_share_directory
import os

def generate_launch_description():
    pkg = get_package_share_directory('oculus_driver')
    params = os.path.join(pkg, 'config', 'slam_toolbox_params.yaml')

    return LaunchDescription([
        DeclareLaunchArgument('sonar_ip', default_value='192.168.2.4'),
        DeclareLaunchArgument('range_m', default_value='10.0'),
        DeclareLaunchArgument('gain', default_value='40.0'),

        # 1) Sonar driver node

```

```
Node(  
    package='oculus_driver',  
    executable='sonar_node',  
    name='oculus_sonar',  
    parameters=[  
        'sonar_ip': LaunchConfiguration('sonar_ip'),  
        'range_m': LaunchConfiguration('range_m'),  
        'gain': LaunchConfiguration('gain'),  
        'salinity': 0.0, # 0=fresh water  
        'sonar_mode': 1,  
    ]  
)  
  
# 2) Static TF  
Node(  
    package='oculus_driver',  
    executable='tf_static',  
    name='static_tf',  
)  
  
# 3) SLAM Toolbox  
Node(  
    package='slam_toolbox',  
    executable='async_slam_toolbox_node',  
    name='slam_toolbox',  
    parameters=[params],  
    output='screen',  
)  
  
# 4) RViz2  
Node(  
    package='rviz2',  
    executable='rviz2',  
    name='rviz2',  
    arguments=['-d', os.path.join(pkg, 'config', 'sonar_slam.rviz')],  
)  
])
```

Register the new executables in `setup.py` entry_points:

```
'console_scripts': [  
    'sonar_node = oculus_driver.sonar_ros_node:main',  
    'tf_static = oculus_driver.tf_broadcaster:main',  
],
```

PYTHON

SECTION 06

Launch & Visualize Everything

Terminal 1

Build everything and launch the full stack:

```
source /opt/ros/humble/setup.bash  
cd ~/ros2_ws  
colcon build --symlink-install  
source install/setup.bash  
  
ros2 launch oculus_driver slam.launch.py \  
    sonar_ip:=192.168.2.4 \  
    range_m:=10.0 \  
    gain:=40.0
```

BASH

Terminal 2

Verify topics are publishing:

```
source ~/ros2_ws/install/setup.bash  
  
# Check all sonar topics are live  
ros2 topic list | grep sonar  
  
# Check point cloud rate (~10Hz expected)  
ros2 topic hz /sonar/pointcloud
```

BASH

```
# Inspect one ping's metadata
ros2 topic echo /sonar/scan --once
```

RViz2 Setup

In the RViz2 window, add these displays:

- **Map** → Topic: `/map` — this is your 2D SLAM map (OccupancyGrid)
- **PointCloud2** → Topic: `/sonar/pointcloud` → Color: `intensity`
- **LaserScan** → Topic: `/sonar/scan`
- **TF** → shows coordinate frames
- Fixed Frame: `map`

Save the Map

When you're happy with the 2D map:

```
# Save occupancy grid as image + YAML
ros2 run nav2_map_server map_saver_cli -f ~/my_sonar_map

# Creates:
#   ~/my_sonar_map.pgm (greyscale map image)
#   ~/my_sonar_map.yaml (resolution, origin metadata)
```

BASH

Using bruce_slam directly (alternative)

If you want the full GTSAM graph from the sonar-SLAM repo instead of slam_toolbox:

```
# Bruce SLAM expects /PointCloud topic + odometry
# Remap our topic:
ros2 launch bruce_slam slam.launch.py \
  __remapping:=/PointCloud:=/sonar/pointcloud
```

BASH

Troubleshooting Reference

SYMPTOM	LIKELY CAUSE	FIX
<code>nc -zv 192.168.2.4</code> 52100 times out	Wrong subnet or sonar IP	Run <code>sudo arp-scan --localnet</code> ; set your PC IP to <code>192.168.2.x</code>
Connected but 0 pings received	Fire message not sending or malformed	Enable debug: check <code>MSG_SIMPLE_FIRE = 0x15</code> ; send a dummy message first
Point cloud looks scrambled	Bearings array misaligned	Check <code>brg_offset = offset + PING_RESULT_SIZE</code> matches struct layout; print <code>bearings_deg.min()/ .max()</code>
LaserScan has zero ranges	Intensity threshold too high	Lower <code>intensity_threshold</code> to <code>0.02</code>
SLAM map not updating	TF tree broken	Run <code>ros2 run tf2_tools view_frames</code> ; ensure <code>map→odom→base_link→sonar_link</code> chain exists
Map drifts badly	No odometry (robot moving)	Add wheel odometry or DVL; or keep sonar static for mapping; increase <code>minimum_travel_distance</code>
<code>gtsam</code> import error	Wrong Python <code>gtsam</code> build	<code>pip3 install gtsam --force-reinstall</code>
Image shape mismatch	M750d sent 16-bit data	Check <code>data_size</code> field; use <code>uint16</code> decode path

Useful Debug Commands

BASH

```
# Record all sonar data to a bag file
ros2 bag record /sonar/pointcloud /sonar/scan /sonar/image -o sonar_bag

# Replay later (without real sonar)
ros2 bag play sonar_bag

# Check sonar image in terminal
```

```
ros2 topic echo /sonar/image --no-arr | head -20
```

TF tree visualization

```
ros2 run tf2_tools view_frames
```

```
evince frames.pdf
```

Print sonar metadata every ping

```
ros2 topic echo /sonar/scan | grep -E "range_min|range_max|angle"
```



Next step after 2D SLAM works: Add real odometry from a DVL or wheel encoders to the TF tree (odom→base_link), then enable the full `bruce_slam` GTSAM graph for loop-closure and drift correction. The driver and point cloud code you built here works unchanged for 3D as well — just add a z component from depth sensor data.